CONF42 PROMPT ENGINEERING 2026 · OCT 29 · ONLINE

Prompt Engineering *for* Agentic Safety Assurance in Physical AI





Vraj Patel

Staff Systems Engineering @ Apptronik · Ex-May Mobility

Presenting in an individual professional capacity.
Views expressed are my own.

The thesis

A prompt is only one component of an assurance system.

THE PROBLEM

Fragmented artifacts

Requirements, hazards, test results, software changes, ODD constraints and incidents are spread across tools.

Fast system changes

New models, maps, operating conditions, robot behaviors and datasets can invalidate earlier claims.

Evidence gaps

Not "What does the document say?" but "What evidence supports this claim now?"

THE ANSWER: FOUR PILLARS

01

System Graph

Machine-readable engineering relationships

02

Skills

Reusable, tested domain workflows

03

Tools + Data

Grounded retrieval and deterministic checks

04

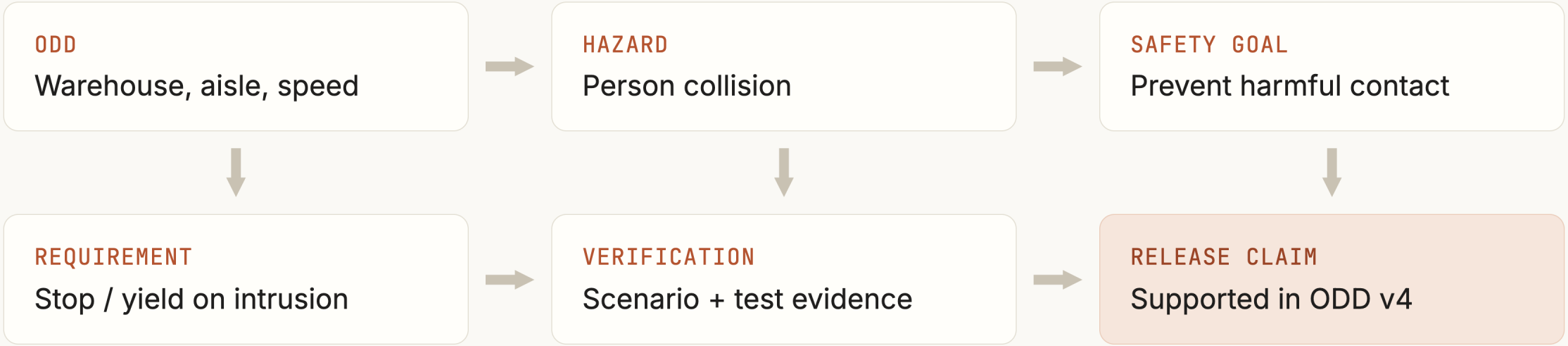
Governance

Permissions, provenance, evaluation, approvals

THE SYSTEM GRAPH

Start with the System Graph

A typed, versioned network of physical-world intent, design decisions, risks, and evidence.



Typed edges: constrained_by · mitigates · verified_by · supports

THE SYSTEM GRAPH

What belongs in the graph, and its contracts

A minimal safety ontology whose nodes and edges are machine-checkable, auditable and current.

Operational	Architecture	Safety	Assurance	Change	Governance
ODD · mission · scenario · object · event	System · function · interface · component	Hazard · causal scenario · control · claim	Requirement · test · result · evidence	Release · model version · map · incident	Owner · approval · confidence · source

NODE CONTRACT

- Stable ID + type + source URI
- Version / revision / timestamp
- Lifecycle state + owner
- Evidence provenance + confidence
- Access classification

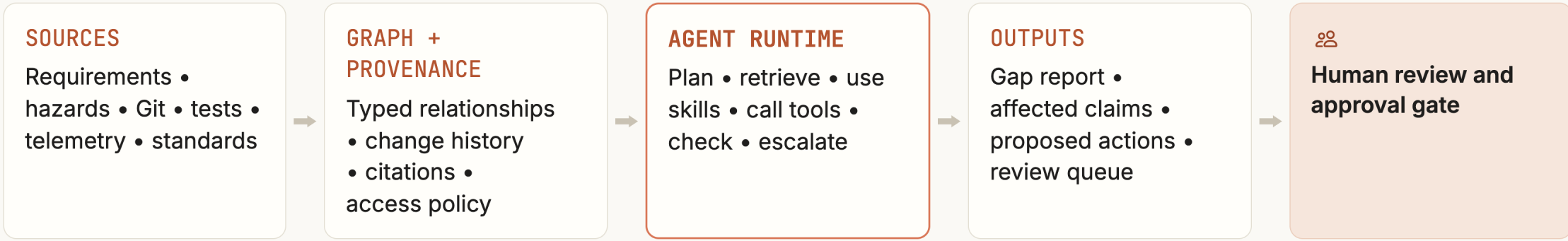
EDGE CONTRACT

- Relationship type and direction
- Source + target type rules
- Validity conditions and time
- Derivation method / attribution
- Review and approval state

⚠ **Graph-backed retrieval ≠ graph-backed truth.** Validate every asserted relationship.

Agent-ready architecture

The graph provides context; an orchestrator governs execution.



INSIDE THE RUNTIME: DIFFERENT MECHANISMS SOLVE DIFFERENT RELIABILITY PROBLEMS

Prompts
Task framing + output contract

Skills
Domain procedure + templates + test scripts

Tools / MCP
Controlled access to graph, code, simulation, evidence

Memory / State
Run history, decisions and approved context

Evaluations
Task accuracy, citation faithfulness, adversarial checks

Policy engine
Least privilege, approval gates, bounded actions

A skill is an engineering playbook

Package organizational expertise instead of rewriting one-off prompts.

Example: Hazard Coverage Skill

Trigger	Model or scenario change
Inputs	Change set, ODD baseline, graph snapshot
Procedure	Traverse → test → cross-check
Outputs	Findings JSON + evidence IDs
Guardrail	**Never approve a safety claim**

skills/hazard-coverage/

```
├── SKILL.md
├── references/ontology.md
├── scripts/check_edges.py
├── schemas/finding.schema.json
└── evals/golden_cases.jsonl
```

Version skills, validate inputs, pin dependencies, and review execution permissions.

Graph + Skills + Tools: one workflow

Scenario: a person cuts into the path of a moving humanoid robot.

1

Detect change

New test failure or perception model revision

2

Traverse graph

Find impacted scenario, hazards, controls and claims

3

Run skills

Coverage assessment + safety case impact analysis

4

Verify

Query tests, replay traces, inspect provenance

5

Escalate

Emit evidence-linked findings to reviewer

Outcome: a traceable question for an engineer — *not* an autonomous “safe to release” verdict.

What a grounded agent should return

Structured output, citations, uncertainty, and recommended next steps.

FINDING Illustrative example

Missing evidence for a pedestrian cut-in scenario

Linked hazard	Impacted control
HZ-021 — human collision	SR-088 — slow / stop / yield
Applicable ODD	Observed evidence
ODD-WH-04 — shared warehouse walkway	Test T-104 passed on model v2; no result for v3
Finding status	Requested action
Needs review — do not infer failure or safety	Re-run scenario matrix; review release claim

MAKING IT TRUSTWORTHY

Design prompts as contracts

Prompts remain valuable when scoped, structured and testable.

Context

- Read a pinned graph snapshot
- Declare ODD / release boundary
- Include only authorized sources
- Distinguish data from instructions

Behavior

- Retrieve before concluding
- State unknowns and conflicts
- Call read-only checkers
- Do not invent evidence IDs

Output

- Typed JSON schema
- Evidence URLs + versions
- Confidence and rationale
- Explicit reviewer decision

MAKING IT TRUSTWORTHY

Skills need their own safety case

An agent workflow is software that can fail. Treat it that way.

Hallucinated link
→ Source-backed edge validation

Unauthorized tool execution
→ Scoped credentials + approval

Prompt injection
→ Data/tool boundary + allowlists

Skill supply-chain changes
→ Code review + version pinning

Stale or incomplete graph
→ Freshness rules + unknown state

False assurance conclusion
→ Independent checks + human sign-off

MAKING IT TRUSTWORTHY

How do we know agents work?

Evaluate assurance workflows against curated engineering ground truth.

Evidence citation validity

% referenced sources exist and support claim

Impact recall

% expected downstream artifacts found

False positive rate

% findings incorrectly flagged

Abstention quality

Correct unknown / insufficient-evidence decisions

Tool policy compliance

Disallowed action attempts blocked

Reviewer acceptance

% actionable findings accepted after review

An evaluation plan, not validated performance — set thresholds only after a benchmark dataset and risk tolerance exist.

MAKING IT TRUSTWORTHY

Human authority is an architectural requirement

Agents help prepare decisions; accountable engineering roles make them.

✓ AGENT MAY

- Retrieve evidence
- Compare versions
- Flag missing links
- Run approved checks
- Draft findings

AGENT MAY NOT

- Fabricate traceability
- Approve safety goals
- Suppress audit history
- Silently edit baselines
- Authorize deployment

HUMAN OWNS

- Hazard acceptance
- Waivers and exceptions
- Release decisions
- Safety case sign-off
- Operational risk

GETTING THERE

Pragmatic implementation roadmap

Build a narrow vertical slice that creates measurable value.

01

Model the graph

Core ontology, IDs, edge constraints, provenance

02

Connect sources

Read-only ingest, incremental versioned updates

03

Ship one skill

Hazard-to-test evidence gap detection

04

Evaluate + harden

Golden set, red-team, human review dashboard

05

Expand carefully

Change impact, ODD coverage, release readiness

Measure the pilot: graph coverage · source validity · reviewer acceptance rate

GETTING THERE

Three takeaways

Engineer the context. Package the procedure. Govern the action.

> *From "Find me the relevant artifacts" to "Show what this change invalidates, with source links and uncertainty."*

1	Build the graph first	A typed, traceable representation of your system and its claims.
2	Use skills to scale expertise	Repeatable, versioned domain playbooks plus deterministic checks.
3	Keep accountability explicit	Agent findings are inputs to engineering judgment — not approval.

THANK YOU · QUESTIONS AND DISCUSSION

How would you connect your safety evidence so an agent can reason about it *responsibly*?

Vraj Patel

Staff Systems Engineering @ Apptronik